

Costas Correlation Phase Estimator

Hilbert FIR + Host-Side atan2 Implementation

EEE299 -- KC705 FPGA Platform

Tony DiMichele

*Xilinx Kintex-7 KC705, AD9627 dual-channel 12-bit ADC @ 125 MSPS,
custom Hilbert FIR phase-shift network, Python host-side atan2.*

1. Introduction

This document describes the FPGA-based Costas correlation phase estimator implemented in `phase_costas.sv`. The module measures the relative phase between two same-frequency sinusoidal signals received on the I (cosine) and Q (sine) channels of an AD9627 dual-channel ADC. The measurement is frequency-independent across the passband (approximately 1.3 MHz to 40 MHz at 125 MSPS sampling).

The system uses a hardware/software co-design approach:

- FPGA: Computes the correlation pair (X, Y) using a 15-tap Hilbert FIR and pipelined accumulators. Normalizes the 40-bit accumulators to 12-bit signed values and transmits them to the host via UDP.
- Host (Python): Receives the normalized X/Y pair and computes $\text{phase} = \text{atan2}(Y, X)$ using IEEE 754 double-precision math. This eliminates FPGA CORDIC IP complexity and provides full-precision four-quadrant phase.

This approach was chosen after validating that FPGA-side CORDIC introduced pipeline-latching artifacts. Moving `atan2` to Python gives exact results with zero FPGA resource cost for the trigonometric computation.

1.1 System Context

The `phase_costas` module is instantiated by `adc_stats.sv`, which orchestrates the adaptive-N zero-crossing frequency measurement, peak tracking, and I-to-Q delay measurement. The `adc_stats` module drives the `window_start` and `window_done` pulses into `phase_costas` based on I-channel zero-crossing events, ensuring the correlation window spans an integer number of signal cycles.

The `adc_stats` output is a 16-byte AXIS byte stream packed as:

Byte	Field
-----	-----
[0]	0xA7 sync byte
[1]	peak_pos[11:4]
[2]	{peak_pos[3:0], peak_neg[11:8]}
[3]	peak_neg[7:0]
[4:7]	iq_delay_sum (unsigned 32-bit big-endian)
[8:9]	measured_clk_count (unsigned 16-bit big-endian)
[10:11]	measured_cycles_count (unsigned 16-bit big-endian)
[12:13]	corr_y (signed 16-bit big-endian, correlation Y)
[14:15]	corr_x (signed 16-bit big-endian, correlation X)

This packet crosses the `adc1_clk` -> `clk_int` clock domain via an async FIFO in `KC705_EEE299_top.v`, then is encapsulated into UDP frames by the ethernet subsystem for delivery to the host.

2. Costas Correlation Phase Estimation

2.1 Mathematical Basis

Consider two sinusoids at the same frequency but with a relative phase offset ϕ :

$$I(t) = A * \cos(\omega t)$$

$$Q(t) = B * \cos(\omega t + \phi)$$

The phase ϕ can be recovered by correlating Q against both I and a 90-degree shifted copy of I (denoted I_{90}). Over one or more complete cycles:

$$X = \text{SUM}[I[n]*Q[n] + I_{90}[n]*Q_{90}[n]] = (AB/2) * \cos(\phi)$$

$$Y = \text{SUM}[I_{90}[n]*Q[n] - I[n]*Q_{90}[n]] = (AB/2) * \sin(\phi)$$

The phase is then:

$$\phi = \text{atan2}(Y, X)$$

This is the standard Costas-loop discriminator formulation. The key advantage is that the amplitude terms A and B cancel in the ratio Y/X , making the measurement amplitude-independent. The atan2 function resolves all four quadrants.

2.2 Windowed Accumulation

The accumulators X and Y run over a measurement window of N positive-going zero crossings of the I channel, where zero-crossing detection uses a hysteresis threshold of ± 4 LSB ($ZC_HYST = 4$ in `adc_stats.sv`). The adaptive- N system in `adc_stats.sv` adjusts N to keep the total clock count in a target range ($FREQ_CLK_TARGET_LO=8000$ to $FREQ_CLK_TARGET_HI=50000$ clocks), bounded by $FREQ_N_MIN=32$ and $FREQ_N_MAX=16384$.

At the window boundary, `adc_stats` pulses `costas_window_done`, which causes `phase_costas` to snapshot the accumulators and pass them to the normalization pipeline. Accumulation for the next window begins immediately. If the accumulator exceeds `FREQ\_ACCUM\_ABORT` (500,000 clocks), `adc_stats` pulses `costas_window_start` instead, which aborts the window without generating output.

2.3 IDDR Channel Alignment

The AD9627 ADC outputs I and Q channels in DDR format. The FPGA IDDR primitive in `SAME_EDGE` mode causes channel B (Q) to arrive 1 sample late relative to channel A (I). This is compensated by delaying `sample_i` by 1 clock in `adc_stats.sv` before feeding it to the phase estimator:

```
reg signed [11:0] sample_i_d1;
always @(posedge clk) sample_i_d1 <= sample_i;

phase_costas u_phase_costas (
    .sample_i(sample_i_d1), // delayed to align with late Q
    .sample_q(sample_q),
    ...
)
```

Without this compensation, all phase readings have a constant offset of $360 * f_{\text{signal}} / f_{\text{sample}}$ degrees (e.g., $+31.7$ degrees at 11 MHz / 125 MSPS).

3. Hilbert FIR Transform (90-Degree Phase Shift)

3.1 Why a Hilbert Filter?

The correlation method requires $I_{90}[n]$, a copy of $I[n]$ shifted by exactly 90 degrees. A BRAM delay line of quarter_period samples introduces frequency-dependent error because the quarter period is rounded to an integer. A Hilbert transform FIR filter provides a true 90-degree phase shift at ALL frequencies within its passband, with a fixed and known group delay, eliminating frequency dependence.

3.2 Filter Design: 15-Tap Type III FIR

The ideal discrete-time Hilbert transform has the impulse response $h[n] = 2/(\pi \cdot n)$ for n odd, 0 for n even. We use a 15-tap windowed truncation (Type III FIR: odd length, antisymmetric). Non-zero coefficients scaled by 2048:

Tap	k (offset)	h_float	h_int (x2048)	RTL param
---	-----	-----	-----	-----
0	-7	-0.0909	-186	H1 = 186
2	-5	-0.1273	-261	H3 = 261
4	-3	-0.2122	-435	H5 = 435
6	-1	-0.6366	-1304	H7 = 1304
7	0	0.0000	0	(center)
8	+1	+0.6366	+1304	H7
10	+3	+0.2122	+435	H5
12	+5	+0.1273	+261	H3
14	+7	+0.0909	+186	H1

All even-offset taps are exactly zero. Antisymmetry means only 4 unique multiplications are needed, using pre-addition of symmetric tap pairs.

3.3 Group Delay and Signal Alignment

A linear-phase FIR of length N has constant group delay $(N-1)/2$ samples. For 15 taps this is 7 samples (GRP_DELAY=7). Both the direct I path and Q signal are delayed by 7 samples via shift registers so that I_{delayed} , $\text{Hilbert}(I)$, Q_{delayed} , and $\text{Hilbert}(Q)$ are all time-aligned.

3.4 DSP48 Efficient Implementation

The Xilinx DSP48E1 pre-adder is exploited for the antisymmetric differences:

```
diff1 = sr[14] - sr[0]    -> * H1 (186)
diff3 = sr[12] - sr[2]    -> * H3 (261)
diff5 = sr[10] - sr[4]    -> * H5 (435)
diff7 = sr[8]  - sr[6]    -> * H7 (1304)
```

This uses 4 DSP48 slices per channel (8 total for I and Q Hilbert). Products are summed into a 27-bit accumulator, right-shifted by 11 (divide by 2048), and saturated to 12-bit signed.

3.5 Frequency Response

A 15-tap Hilbert FIR has passband $\sim 0.04 \cdot f_s$ to $\sim 0.46 \cdot f_s$. At 125 MSPS: ~ 5 MHz to ~ 57 MHz. Below 5 MHz the magnitude rolls off but does NOT affect phase measurement since both X and Y arms are attenuated equally. Measurements remain stable down to ~ 1.3 MHz due to long averaging windows at low frequencies.

4. Normalization and X/Y Output

4.1 Why Normalize?

The correlation accumulators are 40 bits wide (ACC_W=40) to prevent overflow during long measurement windows (up to 50,000 clocks). However, only the ratio Y/X matters for phase. The normalization pipeline scales both X and Y to fit in 12-bit signed range while preserving their ratio, enabling efficient 24-bit UDP transport.

4.2 Two-Stage Pipeline

Split into two registered stages to meet 125 MHz timing:

- Stage 5a: Compute $|\text{snap}_x|$ and $|\text{snap}_y|$ via conditional negation. Select $\max(|X|, |Y|) = \text{norm_max_abs}$.
- Stage 5b: Priority-encode the leading one of norm_max_abs via a 40-bit cascaded ternary chain (bits [39] down to [11]) to produce a 6-bit right-shift amount. This maps the largest accumulator into [0, 2047].

Finally (Stage 6): Apply the arithmetic right-shift to both X and Y, saturate each to [-2048, +2047], and register the outputs as xy_x_out and xy_y_out .

4.3 Host-Side atan2

The host Python analyzer receives the 12-bit signed X and Y values (sign-extended to 16-bit in the UDP packet) and computes:

```
import math
phase_deg = math.degrees(math.atan2(corr_y, corr_x))
```

This gives full IEEE 754 double-precision atan2 with all four quadrants resolved. No CORDIC IP is needed on the FPGA, saving logic resources and eliminating pipeline latching artifacts that were observed with the Xilinx CORDIC IP.

4.4 Advantages Over FPGA CORDIC

- Zero FPGA resource usage for trigonometry (no CORDIC IP instantiation).
- Full double-precision result (vs 16-bit fixed-point CORDIC output).
- No pipeline latency or handshake complexity between CORDIC and output latch.
- Simpler RTL: fewer state machines, no `cordic_pending` flag logic.
- Debugging: raw X/Y visible in packets, allowing immediate verification of correlation quality without ILA.

5. Pipeline Structure and Timing

The full pipeline from ADC input to X/Y output:

Clock	Stage	Operation
0	0	DC-block + shift register load
1	1a	Hilbert pre-add (antisymmetric differences)
2	1b	Hilbert multiply (DSP48)
3	2	Hilbert sum + saturate + register I90/Q90/I_del/Q_del
4	3	Correlation multiply: $I*Q$, $I90*Q$, $I90*Q90$, $I*Q90$
5	4	Accumulate $X += I*Q + I90*Q90$, $Y += I90*Q - I*Q90$
...		(runs for N zero-crossing cycles)
N+5	4	Snapshot: <code>snap_x</code> , <code>snap_y</code> <- final accumulators
N+6	5a	Absolute value + max selection
N+7	5b	Leading-one detect (priority encode)
N+8	6	Shift + saturate + output <code>xy_x_out</code> , <code>xy_y_out</code> , <code>xy_valid</code>

The `window_done` and `window_start` pulses are pipelined through a 5-bit shift register to arrive at the accumulator input exactly when the corresponding products are ready. This ensures the final clock's products are included in the snapshot sum.

5.1 Latency Budget

From `window_done` pulse to `xy_valid` output: 5 (pipeline) + 3 (normalization) = 8 clock cycles = 64 ns at 125 MHz. This is negligible compared to the measurement window duration (typically 8,000-50,000 clocks = 64-400 μ s).

6. DC Blocking Filter

A first-order IIR DC blocker removes any DC offset from both I and Q channels before the Hilbert FIR and correlation. This prevents DC offsets from biasing the accumulator towards zero angle.

Transfer function (z-domain):

$$H(z) = 1 - z^{-1} * (1 - 2^{-K})$$

With DC_BLOCK_SHIFT=8 (K=8), the -3 dB corner is approximately $f_s / (2\pi \cdot 2^K) = 125\text{e}6 / (2\pi \cdot 256) = 77.7\text{ kHz}$. This is well below the minimum signal frequency (~1.3 MHz) so signal attenuation is negligible.

7. FPGA Resource Usage

Estimated resource usage for phase_costas (Kintex-7):

DSP48E1	12	8 Hilbert (4 per I/Q) + 4 correlation
Flip-Flops	~600	Shift regs, pipeline, accumulators
LUT	~400	Normalization, saturation, muxing
BRAM	0	No block RAM needed

Removing the CORDIC IP saves approximately 1 DSP48, ~200 FFs, and ~150 LUTs that were previously used for the iterative rotation pipeline.

8. Measured Results

Measured phase accuracy with 11 MHz, 3 Vpp sinusoidal input, channel alignment compensated (1-sample I delay for IDDR SAME_EDGE Q lag):

40°	+1337	+1579	40.3°	+0.3°
50°	+1588	+1322	50.2°	+0.2°
60°	+1792	+1026	60.2°	+0.2°
70°	+1940	+699	70.2°	+0.2°
80°	+2029	+351	80.2°	+0.2°
90°	+1029	-3	90.2°	+0.2°
100°	+2022	-364	100.2°	+0.2°
110°	+1925	-709	110.3°	+0.3°

Maximum error: 0.3 degrees across 70 degrees of phase sweep. The systematic ~0.2 degree bias is within the Hilbert FIR quantization error at 11 MHz (gain ~ 1.07 vs ideal 1.0).

9. Vpp Frequency Calibration

The measured peak-to-peak voltage exhibits a frequency-dependent attenuation through the ADC signal chain (FMC connector, PCB traces, AD9627 sample-and-hold). This is characterized by comparing FPGA-reported Vpp against a calibrated oscilloscope at fixed 3 Vpp output from the UTG962 signal generator.

Frequency (MHz)	Scope Vpp (V)	FPGA Vpp (V)	Correction Factor
1.5	3.10	2.97	1.044
5	3.10	2.94	1.054
10	3.10	2.90	1.069
15	3.10	2.84	1.092
18	3.10	2.79	1.111
25	3.04	2.56	1.188
32	2.90	2.51	1.155
40	2.72	2.29	1.188

A linear least-squares fit to the correction factor (scope/FPGA) vs frequency gives:

$$\text{cal}(f) = 1.0377 + 0.00409 * f_{\text{MHz}}$$

This is applied in the Python host analyzer to all Vpp and V²_{rms} readings. With calibration enabled, maximum residual error is 4.0% (at 25 MHz where the signal generator itself begins to roll off), and most points are within 1%.

The calibration can be disabled with `--no-vpp-cal` or overridden with `--vpp-cal-a` and `--vpp-cal-b` command-line flags.

10. Python Host Interface

The host-side decoder (`adc_stats_analyzer.py`) processes the 16-byte stat packets:

```
@staticmethod
def _decode_corr_xy(stat_bytes: bytearray) -> tuple:
    raw_y = (stat_bytes[11] << 8) | stat_bytes[12]
    if raw_y >= 0x8000:
        raw_y -= 0x10000
    raw_x = (stat_bytes[13] << 8) | stat_bytes[14]
    if raw_x >= 0x8000:
        raw_x -= 0x10000
    return raw_y, raw_x
```

Phase computation:

```
corr_y, corr_x = self._decode_corr_xy(stat_bytes)
if corr_y != 0 or corr_x != 0:
    phase_deg = math.degrees(math.atan2(corr_y, corr_x))
```

Additional filtering includes EMA smoothing, jump rejection, and optional pi-ambiguity resolution for wrapping near +/-180 degrees.

11. References

- [1] J.P. Costas, 'Synchronous Communications', Proc. IRE, 1956.
- [2] F.M. Gardner, 'Phaselock Techniques', 3rd ed., Wiley, 2005.
- [3] A.V. Oppenheim & R.W. Schaffer, 'Discrete-Time Signal Processing', 3rd ed., Pearson, 2009. Ch. 12: Hilbert Transform.
- [4] S.L. Marple, 'Computing the Discrete-Time Analytic Signal via FFT', IEEE Trans. Signal Processing, 1999.
- [5] Xilinx UG479: 7 Series DSP48E1 Slice User Guide.